

- [1 Comparación de Sintaxis: Gecode vs GNUBison](#)
 - [1.1 Tabla de Contenidos](#)
 - [1.2 1. Introducción](#)
 - [1.2.1 1.1 Propósito](#)
 - [1.2.2 1.2 Diferencias Fundamentales](#)
 - [1.2.3 1.3 Convenciones de este Documento](#)
 - [1.3 2. Sintaxis Gecode](#)
 - [1.3.1 2.1 Estructura General](#)
 - [1.3.2 2.2 Sección de Variables](#)
 - [1.3.3 2.3 Sección de Restricciones \(Constraints\)](#)
 - [1.3.4 2.4 Sección de Funciones](#)
 - [1.3.5 2.5 Ejemplo Completo Gecode](#)
 - [1.4 3. Sintaxis GNUBison](#)
 - [1.4.1 3.1 Estructura General](#)
 - [1.4.2 3.2 Sección de Variables](#)
 - [1.4.3 3.3 Sección de Expresiones](#)
 - [1.4.4 3.4 Formato de Salida](#)
 - [1.4.5 3.5 Ejemplo Completo GNUBison](#)
 - [1.5 4. Comparación Detallada](#)
 - [1.5.1 4.1 Tabla Comparativa: Campos de Variables](#)
 - [1.5.2 4.2 Tabla Comparativa: Tipos de Datos](#)
 - [1.5.3 4.3 Tabla Comparativa: Valores](#)
 - [1.5.4 4.4 Tabla Comparativa: Expresiones](#)
 - [1.5.5 4.5 Tabla Comparativa: Estructura General](#)
 - [1.6 5. Gramática Formal](#)
 - [1.6.1 5.1 Gramática Gecode \(EBNF\)](#)
 - [1.6.2 5.2 Gramática GNUBison \(EBNF\)](#)
 - [1.7 6. Transformaciones](#)
 - [1.7.1 6.1 Transformación Gecode → GNUBison](#)
 - [1.7.2 6.2 Algoritmo de Transformación](#)
 - [1.8 7. Ejemplos Completos](#)
 - [1.8.1 7.1 Ejemplo: Sistema de Control de Acceso](#)
 - [1.8.2 7.2 Ejemplo: Sistema de Monitoreo de Temperatura](#)
 - [1.9 8. Referencia Rápida](#)
 - [1.9.1 8.1 Gecode - Cheat Sheet](#)
 - [1.9.2 8.2 GNUBison - Cheat Sheet](#)
 - [1.9.3 8.3 Tabla de Transformación Rápida](#)
 - [1.10 9. Apéndices](#)
 - [1.10.1 9.1 Apéndice A: Operadores Completos](#)
 - [1.10.2 9.2 Apéndice B: Funciones Disponibles](#)
 - [1.10.3 9.3 Apéndice C: Códigos de Error](#)
 - [1.10.4 9.4 Apéndice D: Herramientas de Conversión](#)
 - [1.10.5 9.5 Apéndice E: Recursos Adicionales](#)
 - [1.11 Glosario](#)

1 Comparación de Sintaxis: Gecode vs GNUBison

Documento de Referencia Técnica Versión: 1.0 **Fecha:** Febrero 2026 **Autores:** Pipeline CSP - Motor Lógico Tipado Multidominio

1.1 Tabla de Contenidos

1. [Introducción](#)
2. [Sintaxis Gecode](#)
3. [Sintaxis GNUBison](#)
4. [Comparación Detallada](#)
5. [Gramática Formal](#)
6. [Transformaciones](#)
7. [Ejemplos Completos](#)

- 8. [Referencia Rápida](#)
- 9. [Apéndices](#)

1.2 1. Introducción

1.2.1 1.1 Propósito

Este documento proporciona una comparación exhaustiva entre dos formatos de especificación de problemas lógicos:

- **Gecode**: Formato de entrada para el pipeline CSP (Constraint Satisfaction Problems)
- **GNUBison**: Formato de evaluación de expresiones con soporte de incertidumbre

1.2.2 1.2 Diferencias Fundamentales

Aspecto	Gecode	GNUBison
Propósito	Definir y resolver CSP	Evaluar expresiones con valores concretos
Paradigma	Solver de restricciones	Evaluador de expresiones
Incertidumbre	No soporta	Soporta múltiples valores simultáneos
Formato	JSON con AST de restricciones	JSON con variables y expresiones
Salida	Soluciones que satisfacen restricciones	Evaluación de verdad de expresiones

1.2.3 1.3 Convenciones de este Documento

- **Código en monospace**: ejemplo
- **Palabras clave en negrita**: **boolean**, **integer**
- **Variables en MAYÚSCULAS**: TEMPERATURA, ZONA
- **Tipos en cursiva**: *logic*, *set*, *float*

1.3 2. Sintaxis Gecode

1.3.1 2.1 Estructura General

El formato Gecode usa JSON con tres secciones principales:

```
{
  "variables": [ ... ],
  "functions": [ ... ],
  "constraints": [ ... ]
}
```

1.3.2 2.2 Sección de Variables

1.3.2.1 2.2.1 Estructura de Variable

```
{
  "name": "NOMBRE_VARIABLE",
  "type": "tipo",
  "domain": [ valores_posibles ],
  "value": [ valores_iniciales ],
  "id": identificador_numérico
}
```

1.3.2.2 2.2.2 Tipos de Variables

1.3.2.2.1 Tipo: boolean

Descripción: Variable lógica binaria (verdadero/falso)

Representación interna: Integer 0/1 (false=0, true=1)

Ejemplo:

```
{
  "name": "PUERTA_ABIERTA",
```

```

    "type": "boolean",
    "domain": [true, false],
    "value": [false],
    "id": 0
  }

```

Interpretación: - domain: Valores posibles (siempre [true, false] para booleanos) - value: Valores iniciales permitidos (subset del dominio) - Internamente Gecode usa 0 (false) y 1 (true)

1.3.2.2.2 Tipo: integer

Descripción: Variable entera con dominio discreto

Representación interna: Integer nativo

Ejemplo:

```

{
  "name": "INTENTOS",
  "type": "integer",
  "domain": [0, 1, 2, 3, 4, 5],
  "value": [0, 1],
  "id": 8
}

```

Interpretación: - domain: Lista explícita de valores enteros permitidos - value: Subset del dominio como valores iniciales - No requiere que el dominio sea continuo

1.3.2.2.3 Tipo: numeric

Descripción: Variable numérica de punto flotante

Representación interna: Integer escalado $\times 1000$

Ejemplo:

```

{
  "name": "TEMPERATURA_C",
  "type": "numeric",
  "domain": [10.0, 40.0],
  "value": [20.0, 25.0],
  "id": 10
}

```

Interpretación: - domain: [mínimo, máximo] (rango continuo) - value: [mín_inicial, máx_inicial] (subset del dominio) -

Importante: Valores se escalan $\times 1000$ internamente - 20.0 $\rightarrow$ 20000 (interno) - 25.5 $\rightarrow$ 25500 (interno)

1.3.2.2.4 Tipo: set

Descripción: Variable de conjunto con elementos etiquetados

Representación interna: Integer (índice en el label map)

Ejemplo:

```

{
  "name": "ZONA",
  "type": "set",
  "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
  "value": ["NORTE", "SUR"],
  "id": 5
}

```

Interpretación: - domain: Array de strings (etiquetas/labels) - value: Subset del dominio - Internamente usa índices: NORTE=0, SUR=1, ESTE=2, OESTE=3

Label Map:

```

NORTE  $\rightarrow$  0
SUR    $\rightarrow$  1
ESTE   $\rightarrow$  2
OESTE  $\rightarrow$  3

```

1.3.3 2.3 Sección de Restricciones (Constraints)

1.3.3.1 2.3.1 Estructura de Constraint

```
{
  "id": número_identificador,
  "expr": "expresión_textual",
  "root": id_nodo_raíz,
  "nodes": [ árbol_sintaxis_abstracta ],
  "var_refs": [ ids_variables_referenciadas ],
  "func_refs": [ ids_funciones_referenciadas ]
}
```

1.3.3.2 2.3.2 Formato de Expresiones

Las expresiones en Gecode se almacenan de dos formas: 1. **Textual** (campo expr): String legible 2. **AST** (campo nodes): Árbol de sintaxis abstracta

Ejemplo completo:

```
{
  "id": 10,
  "expr": "INTENTOS + NIVEL_ALERTA <= 4",
  "root": 4,
  "nodes": [
    {
      "id": 0,
      "type": "Variable",
      "name": "INTENTOS"
    },
    {
      "id": 1,
      "type": "Variable",
      "name": "NIVEL_ALERTA"
    },
    {
      "id": 2,
      "type": "Add",
      "left": 0,
      "right": 1
    },
    {
      "id": 3,
      "type": "Number",
      "value": 4
    },
    {
      "id": 4,
      "type": "LessEq",
      "left": 2,
      "right": 3
    }
  ],
  "var_refs": [8, 9],
  "func_refs": []
}
```

1.3.3.3 2.3.3 Tipos de Nodos AST

Tipo	Descripción	Campos
Variable	Variable del problema	name
Number	Literal numérico	value
Equals	Comparación =	left, right
NotEquals	Comparación !=	left, right
Less	Comparación <	left, right
Greater	Comparación >	left, right
LessEq	Comparación <=	left, right
GreaterEq	Comparación >=	left, right
And	Conjunción lógica	left, right
Or	Disyunción lógica	left, right
Not	Negación lógica	left
Add	Suma aritmética	left, right
Subtract	Resta aritmética	left, right

Multiply	Multiplicación	left, right
Divide	División	left, right
Negate	Negación aritmética	left
In	Pertenencia	left, right
Set	Conjunto de valores	elements[]
Interval	Rango numérico	lo, hi, lo_open, hi_open
FunctionCall	Llamada a función	name, args[]

1.3.3.4 2.3.4 Operadores y Sintaxis de Expresiones

Operadores relacionales:

= Igualdad
!= Desigualdad
< Menor que (estricto)
> Mayor que (estricto)
<= Menor o igual
>= Mayor o igual

Operadores lógicos:

AND Conjunción
OR Disyunción
NOT Negación

Operadores aritméticos:

+ Suma
- Resta
* Multiplicación
/ División

Operadores de conjunto:

IN Pertenencia
{...} Conjunto explícito
[lo,hi] Intervalo cerrado
(lo,hi) Intervalo abierto
[lo,hi) Intervalo semi-abierto

Ejemplos de expresiones:

```
# Comparaciones simples
PUERTA_ABIERTA = false
ALARMA_ACTIVADA = true
TEMPERATURA_C > 20.0

# Expresiones lógicas
TARJETA_VALIDA = true AND HORARIO_LABORAL = true
PUERTA_ABIERTA = false OR ALARMA_ACTIVADA = true

# Expresiones aritméticas
INTENTOS + NIVEL_ALERTA <= 4
TEMPERATURA_C * 1.8 + 32.0 >= 68.0

# Pertenencia a conjuntos
ZONA IN {NORTE, SUR, ESTE}
PERFIL IN {EMPLEADO, SUPERVISOR, ADMIN}

# Pertenencia a intervalos
TEMPERATURA_C IN [18.0, 28.0]
INTENTOS IN [0, 3]

# Funciones
abs(TEMPERATURA_C - 22.0) <= 3.0
```

1.3.4 2.4 Sección de Funciones

1.3.4.1 2.4.1 Estructura de Función

```
{
  "name": "nombre_funcion",
  "params": ["param1", "param2"],
  "body": {
    "type": "Return",
    "expr": nodo_ast_expression
```

```
}
}
```

Ejemplo:

```
{
  "name": "es_supervisor",
  "params": ["perfil"],
  "body": {
    "type": "Return",
    "expr": {
      "type": "Or",
      "left": {
        "type": "Equals",
        "left": {"type": "Variable", "name": "perfil"},
        "right": {"type": "Variable", "name": "SUPERVISOR"}
      },
      "right": {
        "type": "Equals",
        "left": {"type": "Variable", "name": "perfil"},
        "right": {"type": "Variable", "name": "ADMIN"}
      }
    }
  }
}
```

1.3.5 2.5 Ejemplo Completo Gecode

```
{
  "variables": [
    {
      "name": "PUERTA_ABIERTA",
      "type": "boolean",
      "domain": [true, false],
      "value": [false],
      "id": 0
    },
    {
      "name": "ZONA",
      "type": "set",
      "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
      "value": ["NORTE", "SUR"],
      "id": 1
    },
    {
      "name": "TEMPERATURA_C",
      "type": "numeric",
      "domain": [10.0, 40.0],
      "value": [20.0, 25.0],
      "id": 2
    },
    {
      "name": "INTENTOS",
      "type": "integer",
      "domain": [0, 1, 2, 3, 4, 5],
      "value": [0, 1],
      "id": 3
    }
  ],
  "functions": [],
  "constraints": [
    {
      "id": 0,
      "expr": "PUERTA_ABIERTA = false",
      "root": 2,
      "nodes": [
        {"id": 0, "type": "Variable", "name": "PUERTA_ABIERTA"},
        {"id": 1, "type": "Variable", "name": "FALSE"},
        {"id": 2, "type": "Equals", "left": 0, "right": 1}
      ],
      "var_refs": [0],
      "func_refs": []
    },
    {
      "id": 1,
      "expr": "ZONA IN {NORTE, SUR, ESTE}",
      "root": 5,
      "nodes": [
        {"id": 0, "type": "Variable", "name": "ZONA"},
        {"id": 1, "type": "Variable", "name": "NORTE"},
        {"id": 2, "type": "Variable", "name": "SUR"},
        {"id": 3, "type": "Variable", "name": "ESTE"},

```

```

        {"id": 4, "type": "Set", "elements": [1, 2, 3]},
        {"id": 5, "type": "In", "left": 0, "right": 4}
    ],
    "var_refs": [1],
    "func_refs": []
},
{
    "id": 2,
    "expr": "TEMPERATURA_C IN [18.0, 28.0]",
    "root": 4,
    "nodes": [
        {"id": 0, "type": "Variable", "name": "TEMPERATURA_C"},
        {"id": 1, "type": "Number", "value": 18},
        {"id": 2, "type": "Number", "value": 28},
        {"id": 3, "type": "Interval", "lo": 1, "hi": 2, "lo_open": false, "hi_open": false},
        {"id": 4, "type": "In", "left": 0, "right": 3}
    ],
    "var_refs": [2],
    "func_refs": []
},
{
    "id": 3,
    "expr": "INTENTOS + 1 <= 4",
    "root": 4,
    "nodes": [
        {"id": 0, "type": "Variable", "name": "INTENTOS"},
        {"id": 1, "type": "Number", "value": 1},
        {"id": 2, "type": "Add", "left": 0, "right": 1},
        {"id": 3, "type": "Number", "value": 4},
        {"id": 4, "type": "LessEq", "left": 2, "right": 3}
    ],
    "var_refs": [3],
    "func_refs": []
}
]
}

```

1.4 3. Sintaxis GNUBison

1.4.1 3.1 Estructura General

```

{
    "precision": número_decimales,
    "variables": [ ... ],
    "expresiones": [ ... ]
}

```

1.4.2 3.2 Sección de Variables

1.4.2.1 3.2.1 Estructura de Variable

```

{
    "nombre": "NOMBRE_VARIABLE",
    "tipo": "tipo",
    "domain": [ valores_posibles ],
    "value": valor_o_valores
}

```

Diferencias clave con Gecode: - Campo "nombre" en lugar de "name" (español vs inglés) - Campo "tipo" en lugar de "type" - value puede ser un **valor único** (sin incertidumbre) o **array de valores** (con incertidumbre) - No tiene campo id

1.4.2.2 3.2.2 Tipos de Variables

1.4.2.2.1 Tipo: logic

Descripción: Variable lógica (equivalente a boolean en Gecode)

Valores permitidos: true, false

Ejemplo sin incertidumbre:

```

{
    "nombre": "activo",
    "tipo": "logic",
    "domain": [true, false],
    "value": true
}

```

```
}
```

Ejemplo con incertidumbre:

```
{
  "nombre": "activo",
  "tipo": "logic",
  "domain": [true, false],
  "value": [true, false]
}
```

Interpretación: La variable puede ser true O false

1.4.2.2.2 Tipo: integer

Descripción: Variable entera

Ejemplo sin incertidumbre:

```
{
  "nombre": "nivel",
  "tipo": "integer",
  "domain": [1, 10],
  "value": 5
}
```

Ejemplo con incertidumbre:

```
{
  "nombre": "nivel",
  "tipo": "integer",
  "domain": [1, 10],
  "value": [3, 7]
}
```

Interpretación: El nivel puede ser 3 O 7 (no valores intermedios)

1.4.2.2.3 Tipo: float

Descripción: Variable de punto flotante

Ejemplo:

```
{
  "nombre": "temperatura",
  "tipo": "float",
  "domain": [10.0, 40.0],
  "value": 20.5
}
```

Con incertidumbre:

```
{
  "nombre": "temperatura",
  "tipo": "float",
  "value": [18.0, 22.0, 25.0]
}
```

1.4.2.2.4 Tipo: set

Descripción: Conjunto de elementos

Ejemplo:

```
{
  "nombre": "zona",
  "tipo": "set",
  "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
  "value": ["NORTE"]
}
```

Con incertidumbre:

```
{
  "nombre": "zona",
  "tipo": "set",
  "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
  "value": ["NORTE", "SUR"]
}
```



```
}
```

Interpretación: La zona puede ser NORTE O SUR

1.4.3 3.3 Sección de Expresiones

1.4.3.1 3.3.1 Formato de Expresiones

En GNUBison, las expresiones son **strings simples**, sin estructura AST:

```
{
  "expresiones": [
    "activo",
    "activo AND validado",
    "nivel > 5",
    "(activo OR validado) AND (nivel <= 8)"
  ]
}
```

1.4.3.2 3.3.2 Sintaxis de Expresiones

Operadores lógicos:

AND	Conjunción
OR	Disyunción
NOT	Negación
IMPLICA	Implicación lógica ($A \rightarrow B$)

Operadores relacionales:

=	Igualdad
!=	Desigualdad
<	Menor que
>	Mayor que
<=	Menor o igual
>=	Mayor o igual

Operadores aritméticos:

+	Suma
-	Resta
*	Multipliación
/	División

Operadores de conjunto:

IN	Pertenencia
{...}	Conjunto explícito
[lo,hi]	Intervalo cerrado

Funciones:

abs(x)	Valor absoluto
sqrt(x)	Raíz cuadrada
sin(x)	Seno
cos(x)	Coseno

1.4.3.3 3.3.3 Ejemplos de Expresiones

Expresiones simples

```
activo
validado
NOT autorizado
```

Expresiones lógicas

```
activo AND validado
activo OR validado OR autorizado
NOT(activo AND validado)
```

Implicación

```
activo IMPLICA validado
(activo AND conectado) IMPLICA (validado AND autorizado)
```

Comparaciones numéricas

```
nivel > 5
intentos < 4
temperatura >= 18.0
```

```
# Expresiones mixtas
activo AND (nivel > 5)
(validado OR autorizado) AND (intentos <= 3)
```

```
# Pertenencia
zona IN {NORTE, SUR}
temperatura IN [18.0, 28.0]
```

1.4.4 3.4 Formato de Salida

GNUBison genera un JSON con resultados de evaluación:

```
{
  "archivo_entrada": "/path/to/input.json",
  "precision": 3,
  "factor": 1000,
  "variables": [
    {
      "nombre": "activo",
      "tipo": "logic",
      "dominio": [true, false],
      "valor": true
    }
  ],
  "expresiones": [
    {
      "expresion": "activo",
      "resultado": true
    },
    {
      "expresion": "activo AND validado",
      "resultado": false
    },
    {
      "expresion": "nivel > 5",
      "resultado": 0
    }
  ],
  "resumen": {
    "total_variables": 3,
    "total_expresiones": 3,
    "errores": 0,
    "valido": true
  }
}
```

Valores de resultado: - true (1): La expresión es verdadera para todos los valores posibles - false (0): La expresión es falsa para todos los valores posibles - 0: Indeterminado (verdadera en algunos casos, falsa en otros)

1.4.5 3.5 Ejemplo Completo GNUBison

```
{
  "precision": 3,
  "variables": [
    {
      "nombre": "activo",
      "tipo": "logic",
      "domain": [true, false],
      "value": true
    },
    {
      "nombre": "validado",
      "tipo": "logic",
      "domain": [true, false],
      "value": [true, false]
    },
    {
      "nombre": "nivel",
      "tipo": "integer",
      "domain": [1, 10],
      "value": [3, 7]
    },
    {
      "nombre": "temperatura",
      "tipo": "float",
      "domain": [10.0, 40.0],
      "value": 22.5
    },
    {
      "nombre": "zona",
      "tipo": "set",

```

```
      "domain": ["NORTE", "SUR", "ESTE", "OESTE"],
      "value": ["NORTE"]
    }
  ],
  "expresiones": [
    "activo",
    "activo AND validado",
    "activo OR validado",
    "NOT activo",
    "nivel > 5",
    "temperatura >= 18.0 AND temperatura <= 28.0",
    "zona IN {NORTE, SUR, ESTE}",
    "(activo AND validado) IMPLICA (nivel > 5)"
  ]
}
```

1.5 4. Comparación Detallada

1.5.1 4.1 Tabla Comparativa: Campos de Variables

Aspecto	Gecode	GNUBison
Campo nombre	"name"	"nombre"
Campo tipo	"type"	"tipo"
Campo dominio	"domain"	"domain"
Campo valor	"value"	"value"
Campo ID	"id" (numérico)	No existe
Idioma campos	Inglés	Español

1.5.2 4.2 Tabla Comparativa: Tipos de Datos

Gecode	GNUBison	Representación Interna	Notas
boolean	logic	Integer 0/1	Gecode usa 0/1, GNUBison usa true/false
integer	integer	Integer nativo	Idéntico
numeric	float	Integer×1000 (Gecode), Float (GNUBison)	Requiere transformación ÷1000
set	set	Integer índice (Gecode), String array (GNUBison)	Requiere label map

1.5.3 4.3 Tabla Comparativa: Valores

Aspecto	Gecode	GNUBison
Formato	Siempre array	Valor único o array
Incertidumbre	No soporta	Soporta (array de valores)
Boolean	[true] 0 [false]	true, false, 0 [true, false]
Integer	[0, 1, 2]	5 o [3, 7]
Numeric/Float	[20.0, 25.0]	22.5 o [18.0, 25.0]
Set	["NORTE", "SUR"]	["NORTE"] o ["NORTE", "SUR"]

1.5.4 4.4 Tabla Comparativa: Expresiones

Aspecto	Gecode	GNUBison
Formato	String + AST completo	Solo string
Operador AND	AND	AND
Operador OR	OR	OR
Operador NOT	NOT	NOT
Implicación	No nativo	IMPLICA
Igualdad	=	=

Pertenencia	IN	IN
Funciones	Limitado	abs, sqrt, sin, cos, etc.

1.5.5 4.5 Tabla Comparativa: Estructura General

Aspecto	Gecode	GNUBison
Secciones	variables, functions, constraints	variables, expresiones
AST	Sí (completo)	No
Funciones	Definibles por usuario	Built-in
Complejidad	Alta (AST detallado)	Baja (strings simples)
Propósito	Definir CSP completo	Evaluar expresiones

1.6 5. Gramática Formal

1.6.1 5.1 Gramática Gecode (EBNF)

```
(* Gramática del formato JSON de Gecode *)

CSPDocument = "{" VariablesSection "," [FunctionsSection ","] ConstraintsSection "}" ;

(* _____ *)
(* Variables *)
(* _____ *)

VariablesSection = '"variables":' "[" [VariableList] "]" ;

VariableList = Variable { "," Variable } ;

Variable = "{"
    '"name":' String ","
    '"type":' VariableType ","
    '"domain":' Domain ","
    '"value":' ValueSpec ","
    '"id":' Number
    "}" ;

VariableType = '"boolean"' | '"integer"' | '"numeric"' | '"set"' ;

Domain = "[" ValueList "]" ;

ValueList = Value { "," Value } ;

Value = Number | String | Boolean ;

ValueSpec = "[" ValueList "]" ;

(* _____ *)
(* Functions *)
(* _____ *)

FunctionsSection = '"functions":' "[" [FunctionList] "]" ;

FunctionList = Function { "," Function } ;

Function = "{"
    '"name":' String ","
    '"params":' "[" [ParamList] "]" " ","
    '"body":' FunctionBody
    "}" ;

ParamList = String { "," String } ;

FunctionBody = "{"
    '"type": "Return" " ","
    '"expr":' ASTNode
    "}" ;

(* _____ *)
(* Constraints *)
(* _____ *)

ConstraintsSection = '"constraints":' "[" [ConstraintList] "]" ;

ConstraintList = Constraint { "," Constraint } ;
```

```

Constraint = "{
    "id":' Number ",
    "expr":' String ",
    "root":' Number ",
    "nodes":' "[" [ASTNodeList] "]" ",
    "var_refs":' "[" [NumberList] "]" ",
    "func_refs":' "[" [NumberList] "]"
}" ;

```

```

ASTNodeList = ASTNode { ", ASTNode } ;

```

```

ASTNode = "{
    "id":' Number ",
    "type":' NodeType ",
    [NodeFields]
}" ;

```

```

NodeType = '"Variable"' | '"Number"' | '"Equals"' | '"NotEquals"' |
'"Less"' | '"Greater"' | '"LessEq"' | '"GreaterEq"' |
'"And"' | '"Or"' | '"Not"' |
'"Add"' | '"Subtract"' | '"Multiply"' | '"Divide"' |
'"Negate"' | '"In"' | '"Set"' | '"Interval"' | '"FunctionCall"' ;

```

```

NodeFields = VariableField | NumberField | BinaryOpFields |
UnaryOpField | SetFields | IntervalFields | FunctionFields ;

```

```

VariableField = '"name":' String ;

```

```

NumberField = '"value":' Number ;

```

```

BinaryOpFields = '"left":' Number ", '"right":' Number ;

```

```

UnaryOpField = '"left":' Number ;

```

```

SetFields = '"elements":' "[" [NumberList] "]" ;

```

```

IntervalFields = '"lo":' Number ", '"hi":' Number ",
'"lo_open":' Boolean ", '"hi_open":' Boolean ;

```

```

FunctionFields = '"name":' String ", '"args":' "[" [NumberList] "]" ;

```

```

NumberList = Number { ", Number } ;

```

```

(* _____ *)
(* Primitivos *)
(* _____ *)

```

```

String = ''' { character } ''' ;

```

```

Number = ["."] digit { digit } [ "." digit { digit } ] ;

```

```

Boolean = "true" | "false" ;

```

```

digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

```

```

character = (* cualquier carácter Unicode válido *) ;

```

1.6.2 5.2 Gramática GNUBison (EBNF)

```

(* Gramática del formato JSON de GNUBison *)

```

```

BisonDocument = "{ PrecisionField ", VariablesSection ", ExpresionesSection }" ;

```

```

(* _____ *)
(* Campos principales *)
(* _____ *)

```

```

PrecisionField = '"precision":' Number ;

```

```

(* _____ *)
(* Variables *)
(* _____ *)

```

```

VariablesSection = '"variables":' "[" [VariableList] "]" ;

```

```

VariableList = Variable { ", Variable } ;

```

```

Variable = "{
    "nombre":' String ",
    "tipo":' BisonType ",
    ["domain":' Domain ",
    "value":' BisonValue
}" ;

```

```
BisonType = '"logic"' | '"integer"' | '"float"' | '"set"' ;

Domain = "[" ValueList "]" ;

ValueList = Value { "," Value } ;

Value = Number | String | Boolean ;

BisonValue = SingleValue | "[" ValueList "]" ;

SingleValue = Number | String | Boolean | "[" StringList "]" ;

StringList = String { "," String } ;

(* _____ *)
(* Expresiones *)
(* _____ *)

ExpresionesSection = '"expresiones":' "[" [ExpressionList]" " ;

ExpressionList = Expression { "," Expression } ;

Expression = String ;  (* Expresión como string simple *)

(* _____ *)
(* Gramática de expresiones (dentro del string) *)
(* _____ *)

(* Esta es la gramática del lenguaje de expresiones *)

ExprString = LogicExpr ;

LogicExpr = CompExpr { ("AND" | "OR" | "IMPLICA") CompExpr } ;

CompExpr = ArithExpr [( "=" | "!=" | "<" | ">" | "<=" | ">=") ArithExpr ] |
           ArithExpr "IN" SetExpr ;

ArithExpr = Term { ("+" | "-") Term } ;

Term = Factor { ("*" | "/") Factor } ;

Factor = Number |
        Identifier |
        "NOT" Factor |
        FunctionCall |
        "(" LogicExpr ")" ;

FunctionCall = Identifier "(" [ArgList]" " ;

ArgList = LogicExpr { "," LogicExpr } ;

SetExpr = "{" IdentifierList "}" |
         "[" Number "," Number "]" ;

IdentifierList = Identifier { "," Identifier } ;

Identifier = letter { letter | digit | "_" } ;

letter = "A" | "B" | ... | "Z" | "a" | "b" | ... | "z" ;

digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

(* _____ *)
(* Primitivos *)
(* _____ *)

String = ''' { character } ''' ;

Number = ["-"] digit { digit } [ "." digit { digit } ] ;

Boolean = "true" | "false" ;

character = (* cualquier carácter Unicode válido, excepto " sin escapar *) ;
```

1.7 6. Transformaciones

1.7.1 6.1 Transformación Gecode → GNUBison

1.7.1.1 6.1.1 Transformación de Campos

Gecode	GNUBison		
"name"	→	"nombre"	
"type"	→	"tipo"	
"domain"	→	"domain"	(igual)
"value"	→	"value"	(igual, pero formato diferente)
"id"	→	(se omite)	

1.7.1.2 6.1.2 Transformación de Tipos

Gecode	GNUBison		Transformación
boolean	→	logic	Cambio de nombre
integer	→	integer	Sin cambios
numeric	→	float	Cambio + escala ÷1000
set	→	set	Sin cambios (pero valores cambian)

1.7.1.3 6.1.3 Transformación de Valores

Boolean:

Gecode: valor interno = 0 o 1
GNUBison: value = false o true

Transformación:
0 → false
1 → true

Numeric/Float:

Gecode: valor interno escalado ×1000
GNUBison: valor float real

Transformación:
20099 → 20.099
25500 → 25.5

Fórmula: gnu Bison_value = gecode_value / 1000.0

Set:

Gecode: valor = índice numérico (0, 1, 2, ...)
GNUBison: value = array de strings

Transformación con Label Map:
Gecode: ZONA = 0
LabelMap: 0 → "NORTE"
GNUBison: {"nombre": "ZONA", "value": ["NORTE"]}

Integer:

Sin transformación:
Gecode: 5 → GNUBison: 5

1.7.1.4 6.1.4 Ejemplo Completo de Transformación

Entrada Gecode (solución):

PUERTA_ABIERTA=0 ZONA=0 TEMPERATURA_C=20099 INTENTOS=2

Datos del grafo Gecode:

```
{
  "variables": [
    {"name": "PUERTA_ABIERTA", "type": "boolean", ...},
    {"name": "ZONA", "type": "set", "domain": ["NORTE", "SUR", "ESTE"], ...},
    {"name": "TEMPERATURA_C", "type": "numeric", ...},
    {"name": "INTENTOS", "type": "integer", ...}
  ]
}
```

Label Map:

ZONA[0] = "NORTE"
ZONA[1] = "SUR"
ZONA[2] = "ESTE"

Salida GNUBison:

```

{
  "precision": 3,
  "variables": [
    {
      "nombre": "PUERTA_ABIERTA",
      "tipo": "logic",
      "value": false
    },
    {
      "nombre": "ZONA",
      "tipo": "set",
      "value": ["NORTE"]
    },
    {
      "nombre": "TEMPERATURA_C",
      "tipo": "float",
      "value": 20.099
    },
    {
      "nombre": "INTENTOS",
      "tipo": "integer",
      "value": 2
    }
  ],
  "expresiones": [
    "PUERTA_ABIERTA = false",
    "ZONA IN {NORTE, SUR, ESTE}",
    "TEMPERATURA_C IN [18.0, 28.0]",
    "INTENTOS <= 3"
  ]
}

```

1.7.2 6.2 Algoritmo de Transformación

```

function TransformGecodeSolutionToGNUBison(
  solution: GecodeSolution,
  graphData: GecodeGraphData
): GNUBisonJSON;
begin
  result.precision := 3;
  result.variables := [];

  for each var in solution do
    bisonVar.nombre := graphData.getVariableName(var.index);
    geType := graphData.getVariableType(var.index);

    case geType of
      'boolean':
        bisonVar.tipo := 'logic';
        bisonVar.value := (var.value = 1) ? true : false;

      'integer':
        bisonVar.tipo := 'integer';
        bisonVar.value := var.value;

      'numeric':
        bisonVar.tipo := 'float';
        bisonVar.value := var.value / 1000.0;

      'set':
        bisonVar.tipo := 'set';
        label := graphData.getLabelForIndex(var.index, var.value);
        bisonVar.value := [label];
    end;

    result.variables.append(bisonVar);
  end;

  result.expresiones := graphData.getAllExpressions();

  return result;
end;

```

1.8 7. Ejemplos Completos

1.8.1 7.1 Ejemplo: Sistema de Control de Acceso

1.8.1.1 7.1.1 Versión Gecode


```

{
  "variables": [
    {
      "name": "PUERTA_ABIERTA",
      "type": "boolean",
      "domain": [true, false],
      "value": [false],
      "id": 0
    },
    {
      "name": "ALARMA_ACTIVADA",
      "type": "boolean",
      "domain": [true, false],
      "value": [false],
      "id": 1
    },
    {
      "name": "TARJETA_VALIDA",
      "type": "boolean",
      "domain": [true, false],
      "value": [true],
      "id": 2
    },
    {
      "name": "HORARIO_LABORAL",
      "type": "boolean",
      "domain": [true, false],
      "value": [true],
      "id": 3
    },
    {
      "name": "PERFIL",
      "type": "set",
      "domain": ["VISITANTE", "EMPLEADO", "SUPERVISOR", "ADMIN"],
      "value": ["EMPLEADO"],
      "id": 4
    },
    {
      "name": "INTENTOS",
      "type": "integer",
      "domain": [0, 1, 2, 3, 4, 5],
      "value": [0, 1],
      "id": 5
    }
  ],
  "functions": [],
  "constraints": [
    {
      "id": 0,
      "expr": "PUERTA_ABIERTA = false",
      "root": 2,
      "nodes": [
        {"id": 0, "type": "Variable", "name": "PUERTA_ABIERTA"},
        {"id": 1, "type": "Variable", "name": "FALSE"},
        {"id": 2, "type": "Equals", "left": 0, "right": 1}
      ],
      "var_refs": [0],
      "func_refs": []
    },
    {
      "id": 1,
      "expr": "ALARMA_ACTIVADA = false",
      "root": 2,
      "nodes": [
        {"id": 0, "type": "Variable", "name": "ALARMA_ACTIVADA"},
        {"id": 1, "type": "Variable", "name": "FALSE"},
        {"id": 2, "type": "Equals", "left": 0, "right": 1}
      ],
      "var_refs": [1],
      "func_refs": []
    },
    {
      "id": 2,
      "expr": "TARJETA_VALIDA = true AND HORARIO_LABORAL = true",
      "root": 6,
      "nodes": [
        {"id": 0, "type": "Variable", "name": "TARJETA_VALIDA"},
        {"id": 1, "type": "Variable", "name": "TRUE"},
        {"id": 2, "type": "Equals", "left": 0, "right": 1},
        {"id": 3, "type": "Variable", "name": "HORARIO_LABORAL"},
        {"id": 4, "type": "Variable", "name": "TRUE"},
        {"id": 5, "type": "Equals", "left": 3, "right": 4},
        {"id": 6, "type": "And", "left": 2, "right": 5}
      ]
    }
  ]
}

```

```

    ],
    "var_refs": [2, 3],
    "func_refs": []
  },
  {
    "id": 3,
    "expr": "PERFIL IN {EMPLEADO, SUPERVISOR, ADMIN}",
    "root": 5,
    "nodes": [
      {"id": 0, "type": "Variable", "name": "PERFIL"},
      {"id": 1, "type": "Variable", "name": "EMPLEADO"},
      {"id": 2, "type": "Variable", "name": "SUPERVISOR"},
      {"id": 3, "type": "Variable", "name": "ADMIN"},
      {"id": 4, "type": "Set", "elements": [1, 2, 3]},
      {"id": 5, "type": "In", "left": 0, "right": 4}
    ],
    "var_refs": [4],
    "func_refs": []
  },
  {
    "id": 4,
    "expr": "INTENTOS IN [0, 3]",
    "root": 4,
    "nodes": [
      {"id": 0, "type": "Variable", "name": "INTENTOS"},
      {"id": 1, "type": "Number", "value": 0},
      {"id": 2, "type": "Number", "value": 3},
      {"id": 3, "type": "Interval", "lo": 1, "hi": 2, "lo_open": false, "hi_open": false},
      {"id": 4, "type": "In", "left": 0, "right": 3}
    ],
    "var_refs": [5],
    "func_refs": []
  }
]
}

```

1.8.1.2 7.1.2 Versión GNUBison (Solución Concreta)

```

{
  "precision": 3,
  "variables": [
    {
      "nombre": "PUERTA_ABIERTA",
      "tipo": "logic",
      "value": false
    },
    {
      "nombre": "ALARMA_ACTIVADA",
      "tipo": "logic",
      "value": false
    },
    {
      "nombre": "TARJETA_VALIDA",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "HORARIO_LABORAL",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "PERFIL",
      "tipo": "set",
      "value": ["EMPLEADO"]
    },
    {
      "nombre": "INTENTOS",
      "tipo": "integer",
      "value": 0
    }
  ],
  "expresiones": [
    "PUERTA_ABIERTA = false",
    "ALARMA_ACTIVADA = false",
    "TARJETA_VALIDA = true AND HORARIO_LABORAL = true",
    "PERFIL IN {EMPLEADO, SUPERVISOR, ADMIN}",
    "INTENTOS IN [0, 3]"
  ]
}

```

1.8.1.3 7.1.3 Versión GNUBison (Con Incertidumbre)

```

{
  "precision": 3,
  "variables": [
    {
      "nombre": "PUERTA_ABIERTA",
      "tipo": "logic",
      "value": [true, false]
    },
    {
      "nombre": "ALARMA_ACTIVA",
      "tipo": "logic",
      "value": [true, false]
    },
    {
      "nombre": "TARJETA_VALIDA",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "HORARIO_LABORAL",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "PERFIL",
      "tipo": "set",
      "value": ["EMPLEADO", "SUPERVISOR"]
    },
    {
      "nombre": "INTENTOS",
      "tipo": "integer",
      "value": [0, 1, 2]
    }
  ],
  "expresiones": [
    "TARJETA_VALIDA = true AND HORARIO_LABORAL = true",
    "PERFIL IN {EMPLEADO, SUPERVISOR, ADMIN}",
    "INTENTOS IN [0, 3]",
    "(PUERTA_ABIERTA = true) IMPLICA (ALARMA_ACTIVA = false)"
  ]
}

```

1.8.2 7.2 Ejemplo: Sistema de Monitoreo de Temperatura

1.8.2.1 7.2.1 Versión Gecode

```

{
  "variables": [
    {
      "name": "SENSOR_ACTIVO",
      "type": "boolean",
      "domain": [true, false],
      "value": [true],
      "id": 0
    },
    {
      "name": "TEMPERATURA_C",
      "type": "numeric",
      "domain": [0.0, 50.0],
      "value": [18.0, 28.0],
      "id": 1
    },
    {
      "name": "VENTILADOR_ON",
      "type": "boolean",
      "domain": [true, false],
      "value": [true, false],
      "id": 2
    },
    {
      "name": "NIVEL_POTENCIA",
      "type": "integer",
      "domain": [0, 1, 2, 3],
      "value": [0, 1, 2, 3],
      "id": 3
    }
  ],
  "functions": [],
  "constraints": [
    {
      "id": 0,
      "expr": "SENSOR_ACTIVO = true",

```

```

    "root": 2,
    "nodes": [
      {"id": 0, "type": "Variable", "name": "SENSOR_ACTIV0"},
      {"id": 1, "type": "Variable", "name": "TRUE"},
      {"id": 2, "type": "Equals", "left": 0, "right": 1}
    ],
    "var_refs": [0],
    "func_refs": []
  },
  {
    "id": 1,
    "expr": "TEMPERATURA_C IN [18.0, 28.0]",
    "root": 4,
    "nodes": [
      {"id": 0, "type": "Variable", "name": "TEMPERATURA_C"},
      {"id": 1, "type": "Number", "value": 18.0},
      {"id": 2, "type": "Number", "value": 28.0},
      {"id": 3, "type": "Interval", "lo": 1, "hi": 2, "lo_open": false, "hi_open": false},
      {"id": 4, "type": "In", "left": 0, "right": 3}
    ],
    "var_refs": [1],
    "func_refs": []
  },
  {
    "id": 2,
    "expr": "TEMPERATURA_C > 25.0 = VENTILADOR_ON = true",
    "root": 6,
    "nodes": [
      {"id": 0, "type": "Variable", "name": "TEMPERATURA_C"},
      {"id": 1, "type": "Number", "value": 25.0},
      {"id": 2, "type": "Greater", "left": 0, "right": 1},
      {"id": 3, "type": "Variable", "name": "VENTILADOR_ON"},
      {"id": 4, "type": "Variable", "name": "TRUE"},
      {"id": 5, "type": "Equals", "left": 3, "right": 4},
      {"id": 6, "type": "Equals", "left": 2, "right": 5}
    ],
    "var_refs": [1, 2],
    "func_refs": []
  }
]
}

```

1.8.2.2 7.2.2 Versión GNUBison

```

{
  "precision": 3,
  "variables": [
    {
      "nombre": "SENSOR_ACTIV0",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "TEMPERATURA_C",
      "tipo": "float",
      "value": 26.5
    },
    {
      "nombre": "VENTILADOR_ON",
      "tipo": "logic",
      "value": true
    },
    {
      "nombre": "NIVEL_POTENCIA",
      "tipo": "integer",
      "value": 2
    }
  ],
  "expresiones": [
    "SENSOR_ACTIV0 = true",
    "TEMPERATURA_C IN [18.0, 28.0]",
    "(TEMPERATURA_C > 25.0) IMPLICA (VENTILADOR_ON = true)",
    "(VENTILADOR_ON = true) IMPLICA (NIVEL_POTENCIA > 0)"
  ]
}

```

1.9 8. Referencia Rápida

1.9.1 8.1 Gecode - Cheat Sheet

Tipos de Variables:

boolean → IntVar [0,1]
integer → IntVar [min,max] + conjunto discreto
numeric → IntVar × 1000
set → IntVar [0,n-1] + label map

Estructura AST:

```
{  
  "id": N,  
  "type": "TipoNodo",  
  "left": id_izq,    // para operadores binarios  
  "right": id_der,   // para operadores binarios  
  "name": "VAR",     // para Variable  
  "value": número,   // para Number  
  "elements": [...] // para Set  
}
```

Operadores Comunes:

Comparación: Equals, NotEquals, Less, Greater, LessEq, GreaterEq
Lógica: And, Or, Not
Aritmética: Add, Subtract, Multiply, Divide, Negate
Conjunto: In, Set, Interval

1.9.2 8.2 GNUBison - Cheat Sheet

Tipos de Variables:

logic → true/false o [true, false]
integer → número o [n1, n2, ...]
float → número o [f1, f2, ...]
set → ["label"] o ["l1", "l2", ...]

Operadores:

Lógicos: AND, OR, NOT, IMPLICA
Relacionales: =, !=, <, >, <=, >=
Aritméticos: +, -, *, /
Conjunto: IN, {...}, [lo, hi]

Funciones Built-in:

abs(x) sin(x) cos(x) tan(x)
sqrt(x) exp(x) log(x) ln(x)

1.9.3 8.3 Tabla de Transformación Rápida

De (Gecode)	A (GNUBison)	Transformación
"name"	"nombre"	Renombrar campo
"type": "boolean"	"tipo": "logic"	Cambiar tipo
"type": "numeric"	"tipo": "float"	Cambiar tipo
valor_int (boolean)	true/false	0→false, 1→true
valor_int (numeric)	valor_float	÷1000
índice (set)	["label"]	Lookup en LabelMap
[valores] (value)	valor único	Extraer único de array

1.10 9. Apéndices

1.10.1 9.1 Apéndice A: Operadores Completos

1.10.1.1 Gecode

Operador	AST	Sintaxis	Textual	Descripción	Ejemplo
Equals	=			Igualdad	X = 5
NotEquals	!=, /=			Desigualdad	X != 0
Less	<			Menor estricto	X < 10
Greater	>			Mayor estricto	X > 0

LessEq	<=	Menor o igual	X <= 100
GreaterEq	>=	Mayor o igual	X >= 18
And	AND	Conjunción	A AND B
Or	OR	Disyunción	A OR B
Not	NOT	Negación	NOT A
Add	+	Suma	X + Y
Subtract	-	Resta	X - Y
Multiply	*	Multiplicación	X * 2
Divide	/	División	X / 3
Negate	- (unario)	Negación aritmética	-X
In	IN	Pertenencia	X IN {1,2,3}
FunctionCall	func(args)	Llamada función	abs(X)

1.10.1.2 GNUBison

Operador	Descripción	Ejemplo	Precedencia
IMPLICA	Implicación lógica	A IMPLICA B	1 (más baja)
OR	Disyunción	A OR B	2
AND	Conjunción	A AND B	3
NOT	Negación	NOT A	4
=, !=	Igualdad, desigualdad	X = 5	5
<, >, <=, >=	Comparación	X > 0	5
IN	Pertenencia	X IN {1,2}	5
+, -	Suma, resta	X + Y	6
*, /	Multiplicación, división	X * 2	7
- (unario)	Negación aritmética	-X	8 (más alta)

1.10.2 9.2 Apéndice B: Funciones Disponibles

1.10.2.1 GNUBison (Built-in)

Función	Descripción	Dominio	Ejemplo
abs(x)	Valor absoluto	$\mathbb{R} \rightarrow \mathbb{R}^+$	abs(-5) = 5
sqrt(x)	Raíz cuadrada	$\mathbb{R}^+ \rightarrow \mathbb{R}^+$	sqrt(9) = 3
sin(x)	Seno	$\mathbb{R} \rightarrow [-1,1]$	sin(0) = 0
cos(x)	Coseno	$\mathbb{R} \rightarrow [-1,1]$	cos(0) = 1
tan(x)	Tangente	$\mathbb{R} \rightarrow \mathbb{R}$	tan(0) = 0
exp(x)	Exponencial (e ^x)	$\mathbb{R} \rightarrow \mathbb{R}^+$	exp(1) ≈ 2.718
log(x)	Logaritmo base 10	$\mathbb{R}^+ \rightarrow \mathbb{R}$	log(100) = 2
ln(x)	Logaritmo natural	$\mathbb{R}^+ \rightarrow \mathbb{R}$	ln(e) = 1

1.10.2.2 Gecode (User-Defined)

Gecode soporta funciones definidas por el usuario mediante la sección functions:

```

{
  "name": "es_adulto",
  "params": ["edad"],
  "body": {
    "type": "Return",
    "expr": {
      "type": "GreaterEq",
      "left": {"type": "Variable", "name": "edad"},
      "right": {"type": "Number", "value": 18}
    }
  }
}
```

1.10.3 9.3 Apéndice C: Códigos de Error

1.10.3.1 Gecode

Código	Descripción
ERR_SYNTAX	Error de sintaxis en JSON
ERR_TYPE_MISMATCH	Tipo de variable incompatible
ERR_UNDEFINED_VAR	Variable no definida
ERR_UNDEFINED_FUNC	Función no definida
ERR_INVALID_DOMAIN	Dominio inválido
ERR_AST_INVALID	Nodo AST malformado

1.10.3.2 GNUBison

Campo Resumen	Descripción
errores: 0	Sin errores
errores: N	N expresiones con error
valido: true	Todas las expresiones válidas
valido: false	Al menos una expresión inválida

1.10.4 9.4 Apéndice D: Herramientas de Conversión

1.10.4.1 Convertir Markdown a PDF

```
# Usando pandoc
pandoc Comparacion_Sintaxis_Gecode_GNUBison.md \
  -o Comparacion_Sintaxis.pdf \
  --pdf-engine=xelatex \
  --toc \
  --toc-depth=3 \
  --number-sections \
  -V geometry:margin=1in \
  -V fontsize=11pt

# Usando wkhtmltopdf (via HTML)
markdown Comparacion_Sintaxis_Gecode_GNUBison.md | \
  wkhtmltopdf - Comparacion_Sintaxis.pdf
```

1.10.5 9.5 Apéndice E: Recursos Adicionales

Documentación Gecode: - docs/pipeline.txt - Guía completa del pipeline - docs/VerifyWithBison.txt - Documentación del verificador - README.md - Guía de inicio rápido

Documentación GNUBison: - /home/rodo/GNUBison/README.md - Manual de GNUBison - /home/rodo/GNUBison/tests/ - Ejemplos de uso

Ejemplos: - gecode/ejemplos/Json_input_*.json - Entradas de ejemplo - gecode/ejemplos/Json_output_*.json - Grafos AST de ejemplo - GNUBison/tests/test_*.json - Casos de prueba

1.11 Glosario

AST (Abstract Syntax Tree): Árbol de sintaxis abstracta. Representación en forma de árbol de la estructura sintáctica de una expresión.

Boolean: Tipo de dato lógico con dos valores posibles: verdadero (true) o falso (false).

Constraint: Restricción o condición que debe satisfacerse en un problema CSP.

CSP (Constraint Satisfaction Problem): Problema de satisfacción de restricciones. Problema definido por variables, dominios y restricciones.

Domain: Conjunto de valores posibles que puede tomar una variable.

Incertidumbre: Capacidad de representar múltiples valores posibles simultáneamente para una variable.

Label Map: Mapeo entre índices numéricos y etiquetas textuales para variables de tipo set.

Node: Nodo en un árbol AST que representa un operador, variable o literal.

Numeric: Tipo de dato de punto flotante escalado internamente como entero $\times 1000$.

Set: Tipo de dato que representa un conjunto de elementos etiquetados.

Solver: Programa que encuentra soluciones a problemas CSP.

Fin del Documento

Documento generado para el proyecto Pipeline CSP Gecode + GNUBison Integration Versión 1.0 - Febrero 2026